

PROYECTO NO. 1 ESTRUCTURA DE DATOS II: MANUAL TÉCNICO Y DE USUARIO

Curso: Estructura de Datos II

Carrera: Ingeniería en Informática y Sistemas - Universidad Rafael Landívar

Estudiante: Angelo Gustavo García Ixcampary **Carne:** 1547225

Lenguaje de Programación: C# (.NET 10.0 Console Application)

Modalidad: Individual

PARTE I: MANUAL TÉCNICO

1. Descripción General de la Arquitectura

El sistema consiste en una aplicación de consola desarrollada en C# orientada a la gestión eficiente de un catálogo de biblioteca. El sistema resuelve el problema real del control de inventarios, préstamos, devoluciones e indexación rápida mediante la integración modular de tres estructuras de datos avanzadas implementadas completamente desde cero (sin el uso de colecciones nativas avanzadas de .NET como `List<T>`, `SortedDictionary` o `PriorityQueue`).

La arquitectura del código se divide en tres capas bien definidas dentro de carpetas lógicas:

- **Models (Modelo de Datos):** Contiene la entidad `Libro` que abstrae los datos fundamentales de cada obra del catálogo.
- **DataStructures (Estructuras de Datos Propias):** Contiene las clases `BPlusTree`, `BPlusTreeNode`, `MaxHeap` y `MinHeap`.
- **Data (Capa de Presentación y Persistencia):** Contiene la clase `libros.csv` que maneja, la lectura/escritura de archivos CSV y el flujo general de ejecución.
- **Program:** El menú principal de ejecución de cada implementación realizada

2. Justificación Técnica de las Estructuras de Datos

Cada estructura de datos dentro del sistema fue seleccionada de acuerdo con las operaciones más frecuentes exigidas por la lógica del dominio, asegurando la menor complejidad algorítmica posible.

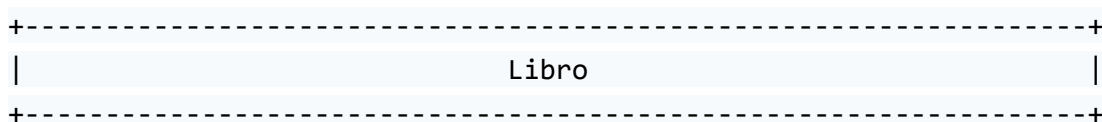
Estructura de Datos	Criterio de Ordenamiento / Clave	Operación Principal Asignada	Complejidad Temporal	Justificación Técnica
Árbol B+ (Grado $m=4$)	Código de Libro (Entero único)	Indexación principal, Búsqueda, Eliminación y Recorrido secuencial completo.	Búsqueda/Insert/Delete: $O(\log n)$ Recorrido general: $O(n)$	Mantiene el catálogo balanceado. A diferencia de un AVL o BST convencional, los datos residen únicamente en los nodos hoja y están enlazados linealmente mediante un puntero <u>Next</u> . Esto permite realizar búsquedas directas por código en $O(\log n)$ y generar listados completos ordenados en $O(n)$ realizando un

Estructura de Datos	Criterio de Ordenamiento / Clave	Operación Principal Asignada	Complejidad Temporal	Justificación Técnica
				simple bucle secuencial sobre las hojas sin recurrir a llamadas recursivas In-Order de alto consumo de memoria.
Max Heap (Montículo Máximo)	VecesPrestado (Entero)	Reporte del Top N de Libros Más Prestados.	Consulta de raíz: $O(1)$ Extracción Max: $O(\log n)$	El nodo raíz siempre contiene el libro con el mayor número de préstamos acumulados. Esto permite acceder al libro más popular en $O(1)$ e iterar secuencialmente para extraer el Top 5 en $O(k \log n)$, siendo $k=5$.
Min Heap (Montículo Mínimo)	CopiasDisponibles (Entero)	Reporte de Libros con Menor Stock / Críticos.	Consulta de raíz: $O(1)$ Extracción Min: $O(\log n)$	La raíz siempre expone el libro con menor número de ejemplares disponibles en

Estructura de Datos	Criterio de Ordenamiento / Clave	Operación Principal Asignada	Complejidad Temporal	Justificación Técnica
				la biblioteca. Permite al personal administrativo detectar en tiempo real $O(1)$ qué libros están agotados (stock 0) para reposición inmediata.
QuickSort Propio	Título (Cadena / String)	Listado secundario ordenado por Título.	Promedio: $O(n \log n)$ Peor caso: $O(n^2)$	Permite reordenar el arreglo extraído del B+ Tree para desplegar el catálogo bajo un criterio alfabético sin alterar el índice físico principal guardado por código.

3. Diagrama de Clases (UML)

El siguiente diagrama estructurado describe las relaciones entre las clases del sistema, el grado de ocultamiento de datos y la herencia de referencias:



+ Codigo: int	
+ Titulo: string	
+ Autor: string	
+ Categoria: string	
+ CopiasDisponibles: int	
+ VecesPrestado: int	
+-----+	
+ Libro(codigo, titulo, autor, categoria, copias, prestado)	
+ ToString(): string	
+-----+	

^	^	
contiene/apunta a	almacena en arreglos	
BPlusTreeNode	MaxHeap / MinHeap	
+-----+		
+ IsLeaf: bool	- heap: Libro[]	
+ NumKeys: int	- capacity: int	
+ Keys: int[]	+ Size: int	
+ ChildrenOrData: obj	+-----+	
+ Next: BPlusTreeNode	+ Insertar(libro: Libro)	
	+ ExtraerMax() / ExtraerMin()	
	+ Buscar(codigo: int): Libro	
	+ Eliminar(codigo: int): bool	
	+ ImprimirHeap(): void	
	+-----+	

+-----+	
BPlusTree	
+-----+	
- order: int = 4	
+ Root: BPlusTreeNode	
+-----+	
+ Buscar(codigo: int): Libro	
+ Insertar(libro: Libro): void	
+ Eliminar(codigo: int): bool	
+ ImprimirCatalogoOrdenado(): void	
+ ImprimirPorTitulo(): void	
+-----+	

4. Diagrama de Flujo Operacional

Flujo A: Carga Masiva desde Archivo CSV e Inserción Múltiple

1. Inicio -> Usuario selecciona Opción 1 en Menú.
2. El sistema solicita la ruta del archivo CSV (Predeterminada: `Data/libros.csv`).
3. ¿Existe el archivo en disco?
 - **NO:** Muestra mensaje de error y regresa al menú.
 - **SI:** Abre la lectura de líneas mediante `File.ReadAllLines`.
4. Descarta el encabezado (Línea 0) y recorre cada línea realizando un `Split(',')`.
5. Instancia un objeto `Libro` con los 6 atributos parseados.
6. Inserta simultáneamente el mismo libro en:
 - `arbolBPlus.Insertar(libro)`
 - `maxHeapPrestados.Insertar(libro)`
 - `minHeapCopias.Insertar(libro)`
7. Notifica el total de registros cargados y retorna al menú.

Flujo B: Registro de Préstamo o Devolución con Persistencia Automática

1. Inicio -> Usuario ingresa el código del libro a prestar o devolver.
2. El sistema invoca `arbolBPlus.Buscar(codigo)` navegando recursivamente en $O(\log n)$.
3. ¿Libro encontrado en la hoja del árbol B+?
 - **NO:** Muestra "Libro no encontrado" y regresa.
 - **SI:** Verifica disponibilidad de copias.
 - Si es **Préstamo:** Si `CopiasDisponibles > 0`, decrementa `CopiasDisponibles--` e incrementa `VecesPrestado++`.
 - Si es **Devolución:** Incrementa `CopiasDisponibles++`.
4. El sistema ejecuta la función `GuardarEnCSV()`: recorre linealmente las hojas enlazadas del Árbol B+ y sobrescribe el archivo `Data/libros.csv` ordenadamente.
5. Muestra mensaje de confirmación.

5. Análisis de Complejidad y Explicación Teórica para

la Defensa

¿Por qué la búsqueda en un Heap es $O(n)$ y no $O(\log n)$?

Los montículos (Heaps) garantizan una relación de orden estrictamente **vertical** entre padres e hijos (Propiedad de Heap). Sin embargo, no poseen un ordenamiento horizontal entre nodos del mismo nivel o entre ramas hermanas. Por esta razón, buscar un elemento dentro de un Heap basado en un atributo arbitrario (como el código de un libro) exige una inspección secuencial del arreglo interno desde la posición 0 hasta `Size-1`, derivando en un costo de tiempo de **$O(n)$** .

¿Cómo evita el sistema la destrucción de los Heaps al consultar rankings?

Dado que la operación `ExtraerMax()` o `ExtraerMin()` altera la estructura física del arreglo removiendo la raíz y reajustando los elementos (`SiftDown`), el programa implementa las funciones auxiliares `ReconstruirMaxHeap()` y `ReconstruirMinHeap()`. Estas funciones recorren las hojas del Árbol B+ y reconstruyen un Heap temporal en tiempo de ejecución. De este modo, los Heaps originales en memoria jamás sufren pérdida de datos.

PARTE II: MANUAL DE USUARIO

1. Requisitos e Instalación del Sistema

- .NET 10.0 SDK instalado en el equipo.
- Terminal de PowerShell, CMD o VS Code.
- Estructura de carpetas requerida en el directorio del proyecto:

```
Proyecto1_biblioteca/  
├── Data/  
│   └── libros.csv  
├── DataStructures/  
│   ├── BPlusTreeNode.cs  
│   └── BPlusTree.cs
```

```
|   |— MaxHeap.cs
|   |— MinHeap.cs
|— Models/
|   |— Libro.cs
|— Program.cs
|— Proyecto1_biblioteca.csproj
```

2. Formato Estándar del Archivo de Datos (.CSV)

El archivo de carga debe ser un documento de texto delimitado por comas ubicado predeterminadamente en `Data/libros.csv` con la siguiente estructura de columnas:

```
Codigo,Titulo,Autor,Categoria,CopiasDisponibles,VecesPrestado
105,Cien Años de Soledad,Gabriel Garcia Marquez,Novela,4,58
101,Don Quijote de la Mancha,Miguel de Cervantes,Clasico,2,72
112,Estructuras de Datos en C#,Luis Joyanes,Tecnico,8,15
```

3. Guía Paso a Paso de las Opciones del Menú Interactivo

Opción 1: Cargar libros desde archivo (.csv)

Permite importar de forma masiva los datos al programa. Al seleccionarla, presione *Enter* para utilizar la ruta por defecto (`Data/libros.csv`) o ingrese una ruta personalizada. El sistema mostrará la cantidad exacta de registros importados exitosamente.

Opción 2: Registrar nuevo libro manualmente

Solicita secuencialmente los 6 campos por consola. Valida en tiempo de ejecución que el código ingresado no exista previamente en el Árbol B+. Una vez validado, el libro se inserta en todas las estructuras y se actualiza el CSV automáticamente.

Opción 3: Buscar libro por código (Árbol B+)

Pide el código entero del libro. El Árbol B+ ejecuta la búsqueda en **$O(\log n)$** e imprime en pantalla el detalle completo del libro si lo localiza.

Opción 4: Eliminar libro (De todas las estructuras)

Solicita el código único y procede a remover físicamente el registro del Árbol B+, del Max Heap y del Min Heap, sincronizando los cambios en el archivo CSV.

Opción 5 y 6: Registrar préstamo / devolución de un libro

Actualiza la cantidad de copias disponibles y las veces prestado. Si no existen copias disponibles de un libro (`CopiasDisponibles = 0`), el sistema bloquea el préstamo e informa al usuario.

Opción 7: Ver catálogo ordenado por CÓDIGO (Árbol B+)

Imprime la totalidad de los libros registrados. El despliegue se realiza recorriendo la lista enlazada de las hojas del Árbol B+, garantizando una salida estrictamente ordenada por código numérico ascendente.

Opción 8: Ver catálogo ordenado por TÍTULO (QuickSort)

Extrae los datos a un arreglo auxiliar en memoria y ejecuta un algoritmo de QuickSort propio por título alfabético, imprimiendo la lista sin alterar el árbol B+ principal.

Opción 9: Ver libros más prestados (Max Heap)

Consulta el Max Heap y despliega en pantalla el Top 5 de libros con mayor cantidad de préstamos registrados en la biblioteca.

Opción 10: Ver libros con menor stock de copias (Min Heap)

Consulta el Min Heap y despliega en pantalla los 5 libros que poseen menor número de copias disponibles para préstamo (ideal para inventario y reposición).

Opción 11: Ver estructuras de Heaps por niveles

Muestra una representación gráfica visual de la distribución en memoria de los arreglos del Max Heap y Min Heap organizados por niveles (`Nivel 0`, `Nivel 1`, `Nivel 2`, etc.).

Opción 12: Guardar cambios y salir

Ejecuta un barrido de confirmación sobre el CSV para asegurar la persistencia en disco y finaliza la ejecución de la consola de forma segura.

Referencias Bibliográficas

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Joyanes Aguilar, L. (2008). *Estructuras de datos en Java*. McGraw-Hill.
- Microsoft. (2024). *C# Guide and .NET Documentation*. Microsoft Learn.